



Παράλληλα Συστήματα
Εργασία Openmp

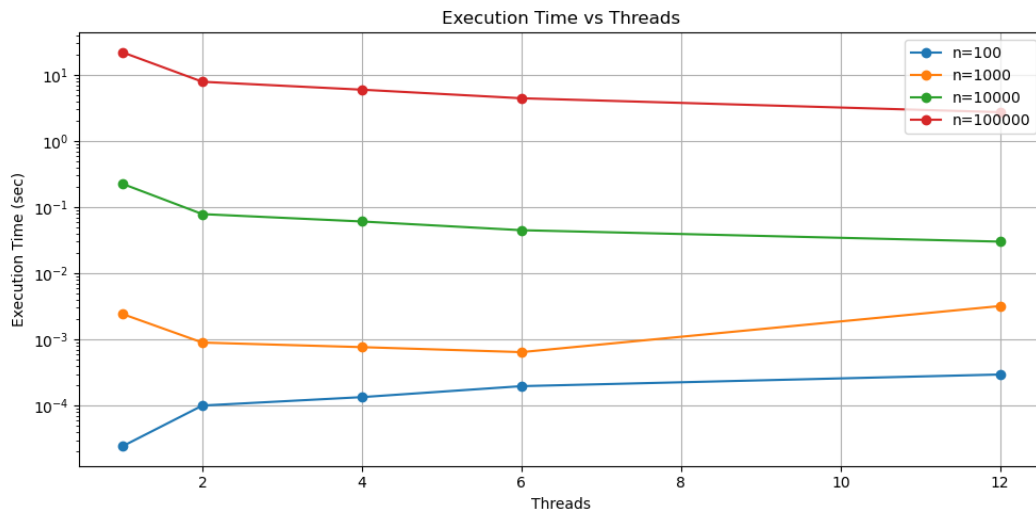
Ονοματεπώνυμο: Ευάγγελος Τραϊκόγλου

Αριθμός Μητρώου: 1115202200189

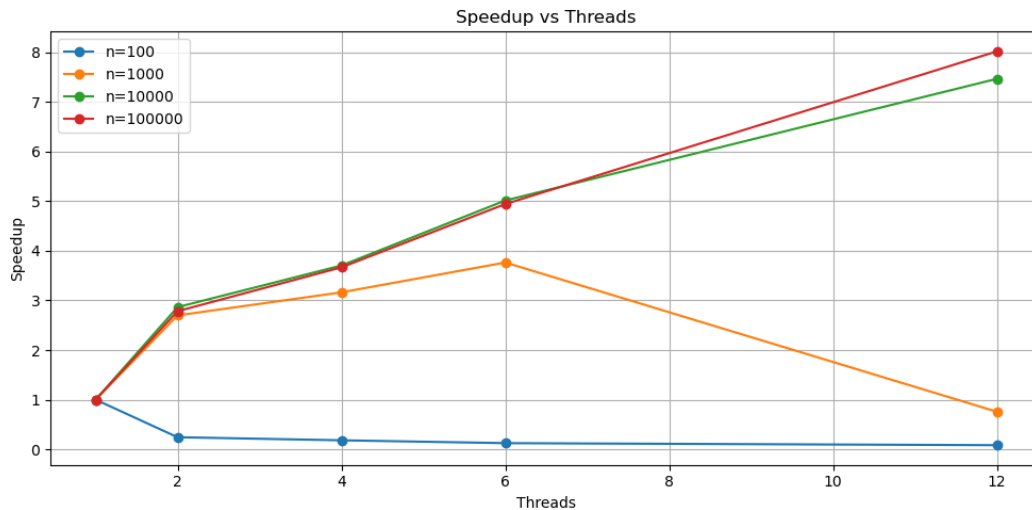
- cpu: i7-8700
- cores: 12 logical, 6 physical
- os, kernel: Ubuntu 24.04.3 LTS, 6.8.0-87-generic
- comp version: gcc 13.3.0

1 Polynomial Multiplication

Η πρώτη άσκηση επικεντρώνεται στον πολλαπλασιασμό πολυωνύμων βαθμού n . Ο σειριακός αλγόριθμος ακολουθεί την απλή υλοποίηση $O(n^2)$ χρόνου. Στον παράλληλο, κάθε νήμα υπολογίζει ένα εύρος του διανύσματος αποτελέσματος σύμφωνα με τον τύπο: $res_k = \sum_{i=0}^k a_i \cdot b_{k-i}$. Αυτό επιτρέπει τη μη χρήση συγχρονισμού αφού κάθε νήμα γράφει σε διαφορετικές θέσεις. Η παράλληλη υλοποίηση χρησιμοποιεί την οδηγία (omp parallel for) για τον βρόχο των k επαναλήψεων και την οδηγία (omp simd reduction(+:sum)) για την simd εκτέλεση του πολλαπλασιασμού στον εσωτερικό βρόχο ($\sim 1\text{sec}$ πιο γρήγορο για ακούρντως μεγάλα πολυώνυμα).



Σχήμα 1: Execution time vs threads for polynomial multiplication



Σχήμα 2: Speedup vs threads for polynomial multiplication

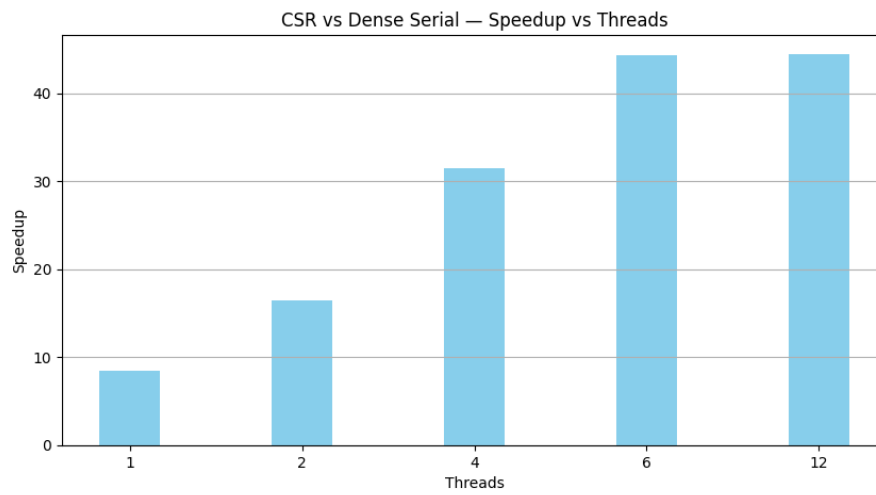
Συμπεράσματα: Για μικρά n (πχ 100), η αύξηση των νημάτων αυξάνει τον χρόνο εκτέλεσης αφού επηρεάζεται κυρίως από την ύπαρξή τους. Κάθως όμως το n αυξάνεται, η αύξηση των νημάτων επιφέρει μείωση του χρόνου εκτέλεσης. Βέβαια το κέρδος από την εισαγωγή ενός νέου νήματος μειώνεται, ακολουθώντας τον νόμο της φθίνουσας απόδοσης. Για $n=1000$, παρατηρείται μια απότομη πτώση στην επίδοση πηγαίνοντας από 6 νήματα σε 12 (όπου λειτουργεί το hyperthreading πλήρως), πιθανώς λόγω της ανεπαρκούς εργασίας ανά νήμα. Αντίστοιχα, η επιτάχυνση (με βάση το σειριακό) μεγιστοποιείται στον μέγιστο αριθμό από νήματα (πέραν της περίπτωσης $n=1000$ όπου μεγιστοποιείται για νήματα=6) και για μεγαλύτερες τιμές του n .

2 Sparse Matrix-Vector Multiplication

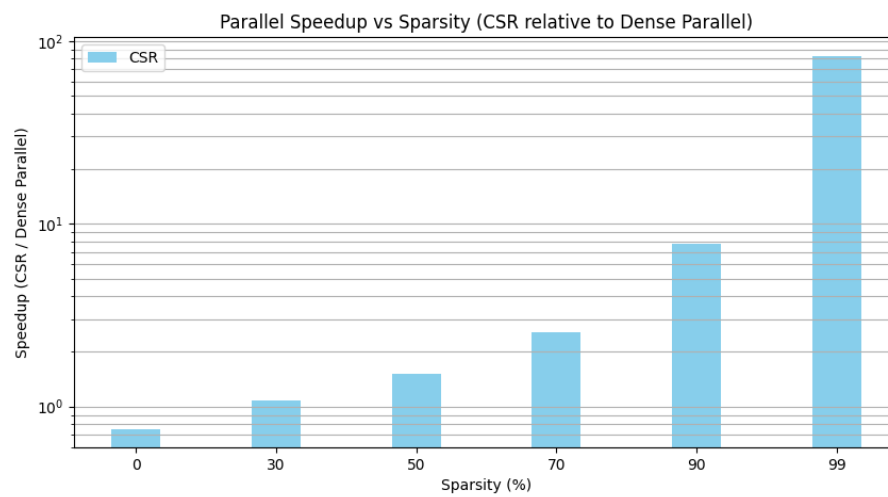
Η δεύτερη άσκηση επικεντρώνεται στον αποδοτικό πολλαπλασιασμό πίνακα-διανύσματος με χρήση της τεχνικής CSR. Η αρχικοποίηση γίνεται ως εξής: Κρατούνται τέσσερις πίνακες: `values`, `col_index`, `row_index`, `row_nz`. Ο `values` κρατά τις μη μηδενικές τιμές του πίνακα με τη σειρά που διαβάζονται (ανά γραμμές). Ο `col_index` τις στήλες που αυτές οι τιμές βρίσκονται (με τη σειρά που εμφανίζονται στον `values`). Ο `row_index` πόσα στοιχεία είναι μη μηδενικά πάνω από τη γραμμή i (πχ `row_index[2]` = πόσα μη μηδενικά υπάρχουν πάνω από τη γραμμή 2). Ο `row_nz` πόσα μη μηδενικά υπάρχουν στη γραμμή i . Πρώτα αρχικοποιείται ο `row_nz` παράλληλα, μετά ένα νήμα υπολογίζει τον `row_index` πίνακα (`row_index[i+1] = row_index[i] + row_nz[i]`). Τέλος, αρχικοποιούνται παράλληλα οι `values`, `col_index`. Η υλοποίηση του πολλαπλασιασμού είναι ακριβώς όπως περιγράφεται στο: <https://people.eecs.berkeley.edu/~aydin/csb2009.pdf>

Δημιουργείται η παράλληλη περιοχή πριν ξεκινήσει ο βρόχος με των `iterations` και ένα μόνο νήμα στο τέλος αντιγράφει το `output vector y` στο `x` για την επόμενη επανάληψη. Αν το `PAR` έχει οριστεί, τρέχει ο `dense parallel` αλγόριθμος αλλιώς ο απλός σειριακός (`dense serial`, default: `PAR`).

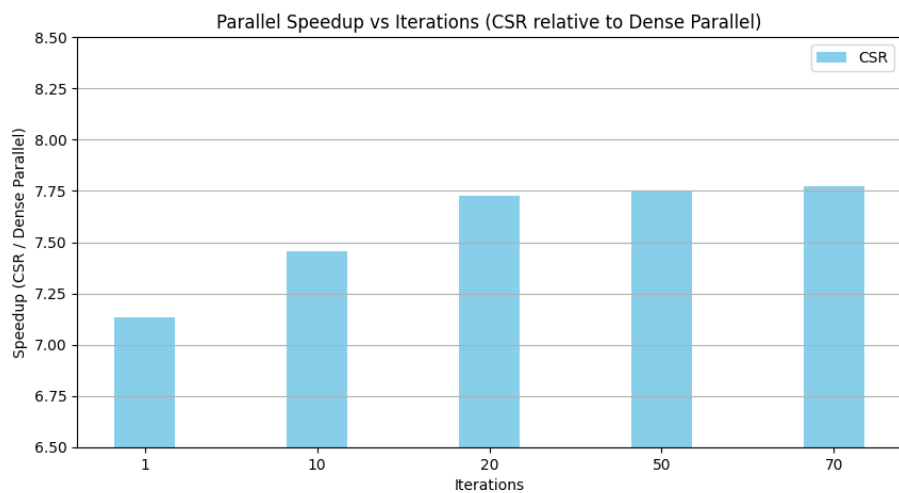
Σημείωση: Όλα τα πειράματα υπάρχουν και στο csv αρχείο: `benchmarks_sparse.csv`. Τα παρακάτω διαγράμματα προβάλλουν κάποια αποτελέσματα για δεδομένες τιμές των άλλων παραμέτρων (που επιλέχθηκαν για να αναδεικνύουν την υπεροχή του CSR).



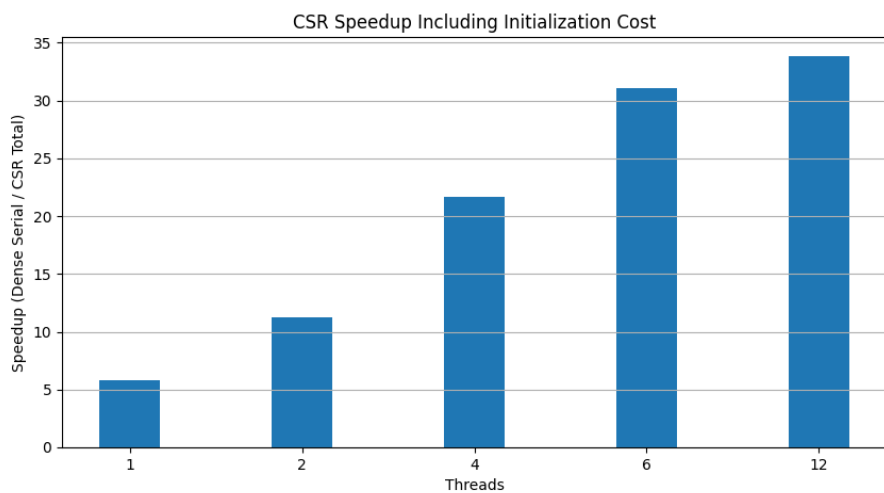
Σχήμα 3: Speedup vs threads: n=10000, sparsity=90, iter=50



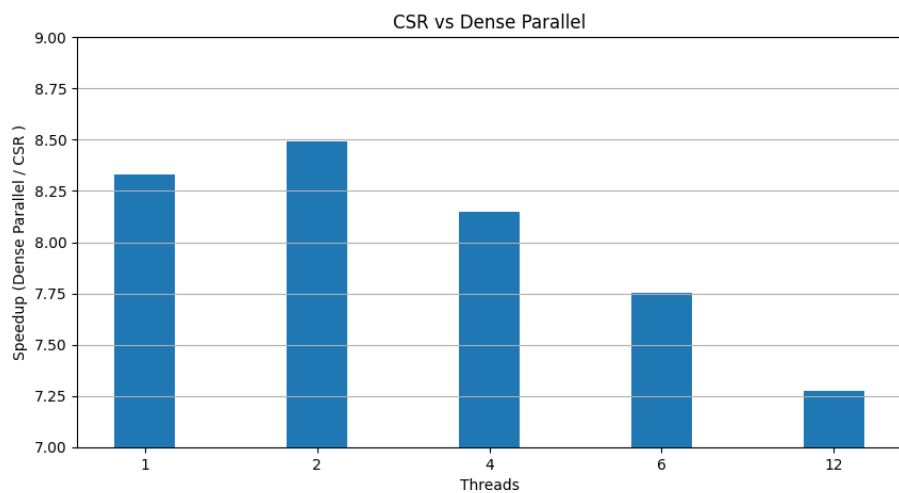
Σχήμα 4: Speedup vs sparsity: n=10000, threads=6, iter=50



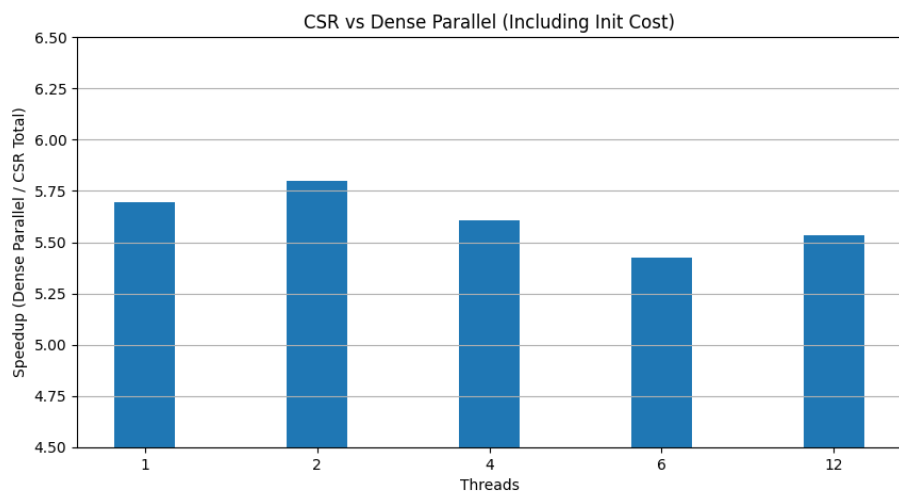
Σχήμα 5: Speedup vs iterations: $n=10000$, sparsity=90, threads=6



Σχήμα 6: Speedup vs threads (Dense serial-CSR total=init+time): $n=10000$, sparsity=90, threads=6



Σχήμα 7: Speedup vs threads (Dense parallel-CSR): $n=10000$, sparsity=90, threads=6



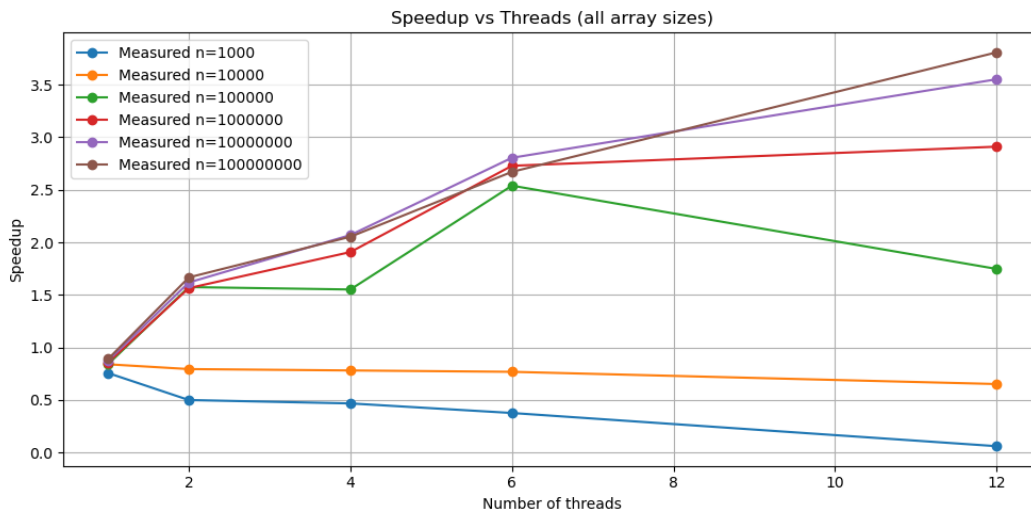
Σχήμα 8: Speedup vs threads (Dense parallel-CSR total=init+time): $n=10000$, sparsity=90, threads=6

Συμπεράσματα: Η επιτάχυνση του CSR σε σχέση με τον σειριακό αλγόριθμο είναι μεγάλη σε υψηλά sparsity και μεγαλύτερους σε μέγεθος πίνακες και προφανώς με την αύξηση των νημάτων γίνεται μεγαλύτερη μέχρι ένα σημείο. Η μετάβαση από τα 6 νήματα στα 12 δεν δίνει θεμιτή επιτάχυνση ακολουθώντας τον νόμο της φθίνουσας απόδοσης. Καθώς αυξάνεται το sparsity και για ακούρντως μεγάλους πίνακες, το CSR υπερτερεί του απλού παράλληλου αλγορίθμου, αφού παραλείπονται οι ανούσιοι πολλαπλασιασμοί. Επιπλέον, η αύξηση του αριθμού των επαναλήψεων επιφέρει αύξηση στην επιτάχυνση καθώς η υπεροχή του CSR δρα αθροιστικά στην αύξηση του πλήθους των επαναλήψεων σε σχέση με την απλή παράλληλη εκδοχή. Ο χρόνος για την αρχικοποίηση του CSR αυξάνεται καθώς το πλήθος των μη μηδενικών στοιχείων αυξάνεται (αφού μεγαλώνουν οι πίνακες values, col_index, non_zr). Άρα η επιτάχυνση του CSR συνολικά (init time + execute time) είναι μικρότερη από την σύγκριση με μόνο το execute time, όπως φαίνεται και στα αντίστοιχα διαγράμματα (3,6, 7,8).

3 Mergesort with tasks

Η τρίτη άσκηση αφορά την αποδοτική υλοποίηση του mergesort με τη χρήση των tasks της Openmp. Η σειριακή υλοποίηση είναι η γνωστή. Η παράλληλη δημιουργεί ένα παράλληλο block πρώτα n νημάτων και ένα από αυτά καλεί την συνάρτηση parallel_mergesort. Μέσα στη συνάρτηση τα tasks δημιουργούνται αν οι υποπίνακες είναι ακούρντως μεγάλοι (> 20000 , όπου η τιμή βρέθηκε μετά από πειράματα). Το νήμα που δημιουργεί τα tasks περιμένει να ολοκληρώσουν μέσω της taskwait.

Σημείωση: Τα πειράματα υπάρχουν και στο csv αρχείο: benchmarks_mergesort.csv.



Σχήμα 9: Speedup vs threads for mergesort

Συμπεράσματα: Η επιτάχυνση που παρατηρείται σε σχέση με τη σειριακή υλοποίηση είναι περιορισμένη. Αφενός λόγω της σειριακής εκτέλεσης του merge βήματος, το οποίο χρειάζεται να αντιγράψει στους δύο καινούργιους υποπίνακες ένα εύρος στοιχείων του αρχικού πίνακα και αντίστοιχα να τους αποδεσμεύσει (αν έχουν δεσμευθεί με malloc). Ο αλγόριθμος δηλαδή είναι memory-bound, για αυτό δεν πετυχαίνεται καλή επιτάχυνση. Σε κάποιες περιπτώσεις, η μετάβαση από τα 6 στα 12 νήματα επιφέρει επιβράδυνση, κυρίως στα μικρότερα μεγέθη πινάκων.